

Uganda and Malta — who can see which data

Same 8 APIs. Two countries. Two databases.

Simple way: Client ID and secret (like a username and password)

More complete way: JWT token (get a pass, then call the API)

The idea in one minute

We have 8 APIs (Invoice, Order, Product, Route, SalesRep, Customer, Stock, Warehouse).

Uganda and Malta will call **the same URLs**. Only the database behind the API is different.

The customer must **not** send the database name. If they could type database=Malta, Uganda could look at Malta data.

Simple rule: We look at *who is calling*. Then **our Experience API** picks the database. Uganda login → Uganda data. Malta login → Malta data.

Think of a hotel. Same front door. Uganda guest key opens Uganda rooms only. Malta key opens Malta rooms only. Nobody is asked “which floor do you want?” — the key already decides.

What the customer sends (both ways)

They send	They do not send
fromDate, toDate, pageNumber	database name
Their login (Client ID or JWT token)	supplier name
Same URL, for example /api/v1/Invoice	region=uganda or region=malta

Option 1 — Simple: Client ID

Give each country a **ID** and a **password** (called `client_id` and `client_secret`).

Example

Country	ID (example)	Password (example)	Database we set
Uganda	ug-client-001	UgSecret-AAA	sat_uganda
Malta	mt-client-002	MtSecret-BBB	sat_malta

Uganda calls Invoice

```
GET /api/v1/Invoice?fromDate=2026-08-01&toDate=2026-08-31&pageNumber=0
client_id:      ug-client-001
client_secret:  UgSecret-AAA
```

We see ID `ug-client-001` → we set database = `sat_uganda`
Uganda gets Uganda invoices only.

Malta calls the same Invoice URL

```
GET /api/v1/Invoice?fromDate=2026-08-01&toDate=2026-08-31&pageNumber=0
client_id:      mt-client-002
client_secret:  MtSecret-BBB
```

We see ID `mt-client-002` → we set database = `sat_malta`
Malta gets Malta invoices only.

If Uganda tries to cheat

```
GET /api/v1/Invoice?database=sat_malta    ← we ignore this
client_id: ug-client-001
```

Still Uganda data. The ID decides, not the URL.

Wrong or missing password

No ID / wrong password → **401 not allowed**. No data.

Easy to explain: two logins, two databases. Same 8 APIs.

Watch out: if Uganda learns Malta's ID and password, they can open Malta data — same as using someone else's hotel key. Keep passwords private. Change them if leaked.

This is the **simple process** for the client. Fast to build. SAT already works this way on many Mule APIs.

Option 2 — More steps: JWT token

JWT is a **short pass** (like a visitor badge that expires in 10 minutes). You do not send the password on every Invoice call. You first get a badge, then you show the badge.

Two steps — example for Uganda

Step 1 — Get a token (once, then reuse until it expires)

```
POST /token
username / client:  ug-client-001
password / secret:  UgSecret-AAA
```

Reply (short example):

```
{
  "region": "uganda",
  "exp":    "10 minutes from now"
}
```

This reply is signed. Nobody can change region to malta without breaking the badge.

Step 2 — Call Invoice with the token

```
GET /api/v1/Invoice?fromDate=2026-08-01&toDate=2026-08-31&pageNumber=0
Authorization: Bearer (the token from step 1)
```

We read region = uganda → we set database = sat_uganda

Malta is the same two steps

Step 1: Malta login → token with region = malta

Step 2: same /api/v1/Invoice + Malta token → database = sat_malta

If someone copies the token

It only works until it **expires** (for example 10 minutes). Then they must login again. That is the extra safety vs a password that stays the same for months.

If someone changes the token text to say “uganda”

The signature will not match. We reject it. **401**. They cannot fake the other country.

Why it is more complex: SAT must call a token URL first, store the token, refresh it when it expires. We also need a token server (Anypoint / Okta / Azure).

Why some clients want it: password is not sent on every request; pass expires; easy to add more countries later.

Side by side

	Simple — Client ID	More steps — JWT
What Uganda shows	ID + password on every call	A token (badge) on every call
How many steps	1 — call the API	2 — get token, then call the API
Same 8 URLs	Yes	Yes
Who picks database	We do, from the ID	We do, from region inside the token
Can they type the other database	No	No
If password is leaked	Other country can call until we change the password	They can get new tokens for that country until we revoke; a copied token dies after a few minutes
Build effort for us	Small	More (token server + policy)
Build effort for SAT	Small	A bit more (get token, refresh)
Best when	Two systems, go live soon	They already use OAuth, or want expiry / more countries later

What we recommend saying in the meeting

- **Start with Client ID** if they want the simple process.
- **Use JWT** if they already have login/tokens, or they want the pass to expire.
- Either way: same 8 APIs. We set the database. They never send it.
- PAPI and SAPI stay internal. Only the Experience API is public.
- Passwords / tokens stay in each country's system. Do not share Uganda login with Malta.

One line for the client:

Simple = two passwords, we pick the database.

JWT = first get a short pass, then call the same APIs. Safer over time, more moving parts.

What does not change

- The 8 endpoint names and the Excel mappings.
- Dates and paging.
- The SQL — only the schema name in front of the table changes (sat_uganda vs sat_malta).

Simple version · SAT Datashare · Uganda / Malta